Spatial Navigation using Grid Cells

Path Integration in 2D Space

NX-465 Mini-project **MP2**

Spring semester 2026

* Read the general instructions carefully before starting the mini-project. *

Introduction

Path integration is a fundamental computational strategy used by animals to keep track of their location by integrating their own self-motion cues. A key component of the brain's navigation system resides in the entorhinal cortex, where "grid cells" fire when an animal crosses the vertices of a remarkably regular, periodic triangular lattice tiling the environment (Hafting et al., 2005). This important discovery was awarded a Nobel Prize in 2014.
Continuous Attractor Network (CAN) models provide a mechanistic explanation for how the brain might generate and update this coordinate framework. This project heavily relies on the grid cell CAN model proposed by Burak and Fiete (2009). Throughout this project, you will implement a recurrent continuum model (Week 6 of lecture) to rigorously simulate grid cells that could enable path integration.
You will start by building a 1D continuous ring attractor that generates periodic bumps, then use it to create a 1D velocity integrator. Later, you will extend these principles into a 2D sheet of neurons capable of sustaining multiple bumps in a hexagonal grid pattern and integrating physical 2D velocity inputs. Finally, you will explore the different boundary conditions and mitigate boundary artifacts using techniques as proposed by Burak and Fiete (2009).

Note: The project is intended to be solved using Python. Functions from `scipy.ndimage` (like `convolve1d` and `convolve`) are helpful for computing the recurrent inputs.

Exercise 0. Getting Started: Ring Attractors in 1D

We begin with a 1D continuous ring attractor model. We simulate a field model integrated via the forward Euler method. The potential $s(x,t)$ of a neuron at position x evolves according to:

$$\tau \frac{ds(x,t)}{dt} = -s(x,t) + I_{rec}(x,t) + B(x,t) \tag{1}$$

where $B(x,t)$ is an external input to the neuron, and the recurrent input $I_{rec}(x,t)$ is defined as the convolution of each neuron's firing rate $r(x,t)$ with a synaptic weight kernel W that depends on the distance between two neurons:

$$I_{rec}(x,t) = \int W(x-x')r(x',t)dx' \tag{2}$$

The firing rate $r(x,t)$ of a neuron at position x is given by a simple ReLU activation on the potential:

$$r(x,t) = \max(0, s(x,t)) \tag{3}$$

The recurrent connectivity kernel $W(x)$ is a Mexican-hat (difference of Gaussians) kernel:

$$W(x) = A_{exc}e^{-x^2/\sigma_{exc}^2} - A_{inh}e^{-x^2/\sigma_{inh}^2} \tag{4}$$

0.1. Write a Python function `mexican_hat_1d(x, A_exc, sigma_exc, A_inh, sigma_inh)` that computes the 1D interaction weight at distance x.

0.2. Plot the weight kernel for an array of $M = 100$ neurons centered around 0 (i.e., $x \in [-\frac{M}{2}, \frac{M}{2})$, equally spaced), using parameters: $A_{exc} = 0.3, \sigma_{exc} = 5.0, A_{inh} = 0.2, \sigma_{inh} = 10.0$.

0.3. Explain which biological feature of cortical connectivity this kernel represents, in terms of excitation and inhibition.

If we were to simply apply this connectivity scheme to M neurons, it is not clear how the neurons at the boundary should be connected to other neurons. Here, we assume the network relies on **periodic boundary conditions**. Mathematically, this means that the connectivity kernel is periodic, i.e. $W(x+M) = W(x)$ for every x. Equivalently, we could imagine that the M neurons lie on a ring.
To simulate the network, we will use the Forward Euler method with a time-step of $\Delta t = 1$ ms:

$$s(x, t_{k+1}) = s(x, t_k) + \Delta t \cdot \frac{ds(x, t_k)}{dt} \tag{5}$$

0.4. Use the forward Euler method with timestep $\Delta t = 1$ ms to integrate the dynamics of this 1D network over 500 ms. Plot the evolution of the firing rate $r(x,t)$ of all neurons. (Suggestions: you could use a heat map to visualize this).
Parameters: $\tau = 10$ ms, $B(x,t) = 1.0$. Randomly initialize the potentials $s(x,0)$ uniformly in $[0, 0.1]$.
Hint: To easily implement the convolution $\int W(x-x')r(x')dx'$ discretely, you could first precompute the connectivity kernel as in Ex. 0.2. and then use `scipy.ndimage.convolve1d`, with proper `mode` to reflect the periodic boundary.

0.5. Describe the behavior of the network. Use 3 different random seeds to initialize the network. Does the behavior depend on the initialization of $s(x,0)$?

Exercise 1. 1D Velocity Integration

To track velocity, we include two sub-populations in the network: a "Right-shifting" population (R) and a "Left-shifting" population (L), each with $M = 100$ neurons. There are two neurons at each location x, one neuron for each population. The recurrent connectivity and external input are defined as follows:

- The connection weights coming from the R population are shifted slightly to the right by a weight shift l, meaning a neuron at x' in R projects to a neuron at x with weight $W_R(x-x') = W(x-x'-l)$.
- The connection weights coming from the L population are shifted to the left by l: $W_L(x-x') = W(x-x'+l)$.
- Each population receives its own external input, B_R and B_L, which is modulated asymmetrically by an external velocity signal $v(t)$:

$$B_R(x,t) = B_0(1+\alpha v(t))$$
$$B_L(x,t) = B_0(1-\alpha v(t))$$

Both populations project to each other as well as to themselves. The total recurrent input received by any neuron (whether it belongs to the R or L population) is the sum of inputs from both populations:

$$I_{rec}(x,t) = \int W_R(x-x')r_R(x',t)dx' + \int W_L(x-x')r_L(x',t)dx' \tag{6}$$

1.1. Write a simulation class or function for this 2-population integrator. Use the previous parameters, but set $B_0 = 1.5$, $\alpha = 0.5$, and weight shift $l = 1.0$. Run the simulation for 1000 ms with a timestep of 1 ms. For the first 300 ms, set $v = 0$ to let the activity pattern stabilize, and you should observe stable bumps. From $t = 300$ ms to 700 ms, apply a constant velocity $v = 1.0$. For $t > 700$ ms, $v = 0$. Plot the sum of the firing rates $r_{total}(x,t) = r_R(x,t) + r_L(x,t)$ over time.

1.2. Explain how the asymmetric scaling of the external inputs leads to shifting bumps in response to a velocity signal, based on the interactions between the two populations.

1.3. To show that the bump acts as a velocity integrator, we apply a velocity signal v of varying magnitudes over time. Sample 10 velocities in the interval $[-2,-0.5] \cup [0.5,2]$. The velocity changes every 300 timesteps. Plot the evolution of the bump and the velocity signal side by side. Describe your observations.

1.4. Devise a method to compute the velocity of the moving 1D bump pattern. Use this method to extract the bump velocity at different input velocities v sampled in 1.3. Plot the bump velocity versus the actual input velocity. Can you see a linear relationship between the two?

Exercise 2. Extending to 2D Grid Cells

We now extend to a 2D sheet of neurons. Each neuron has a 2D position $\vec{\mathbf{x}} = (x,y)$. Then equations (1) and (2) can be directly extended for the 2D case by using $\vec{\mathbf{x}}$ and $\vec{\mathbf{x}}'$ to replace x and x'. The Mexican-hat connectivity in 2D is defined using the Euclidean norm of the vector: $||\vec{\mathbf{x}}|| = \sqrt{x^2+y^2}$.

$$W(\vec{\mathbf{x}}) = A_{exc}e^{-||\vec{\mathbf{x}}||^2/\sigma_{exc}^2} - A_{inh}e^{-||\vec{\mathbf{x}}||^2/\sigma_{inh}^2} \tag{7}$$

2.1. Consider an $M \times M$ array of neurons ($M = 40$, so 1600 neurons), plot the mexican-hat weight kernel $W(\vec{\mathbf{x}})$ centered around 0 in a 2D heatmap (i.e., $x, y \in [-\frac{M}{2}, \frac{M}{2})$, equally spaced). Use parameters: $A_{exc} = A_{inh} = 1.0,\ \sigma_{exc} = 4.8,\ \sigma_{inh} = 5.0$,.

To model movement in the 2D space, we use 4 distinct populations: North (N), South (S), East (E), and West (W), each with 2D Mexican-hat kernels shifted by a weight shift l along their respective preferred axes, as we have done in the 1D scenario:

$$W_N(\vec{\mathbf{x}}) = W(\vec{\mathbf{x}} - (0,l)) \qquad W_S(\vec{\mathbf{x}}) = W(\vec{\mathbf{x}} + (0,l))$$
$$W_E(\vec{\mathbf{x}}) = W(\vec{\mathbf{x}} - (l,0)) \qquad W_W(\vec{\mathbf{x}}) = W(\vec{\mathbf{x}} + (l,0))$$

The feedforward inputs are modulated by a 2D velocity vector $\vec{\mathbf{v}}(t) = (v_x(t), v_y(t))$:

$$B_N = B_0(1 + \alpha v_y(t)) \qquad B_S = B_0(1 - \alpha v_y(t))$$
$$B_E = B_0(1 + \alpha v_x(t)) \qquad B_W = B_0(1 - \alpha v_x(t))$$

2.2. Implement the 2D model. Create an $M \times M$ array of neurons ($M = 40$). Use the above mexican hat kernel as 2D connectivity with periodic boundary (topologically, connecting the boundaries of a 2D sheet means that the neurons lie on a torus). Simulate with $\vec{v} = (0, 0)$ until the pattern stabilizes (e.g., 400 ms). Plot a heatmap of the total 2D firing rate $r_{total}(x, y)$ (sum of all 4 populations) at the final timestep. Do you see a grid pattern? Use parameters: $\tau = 10$ms, $\Delta t = 1$ms, $l = 1.0$, $\alpha = 0.1$, $B_0 = 1.0$.

2.3. To test that your grid maps act as a continuous 2D integrator, apply constant velocity vectors in at least 3 different directions (e.g., purely horizontal, purely vertical, and diagonal). For each velocity direction, first run your network without any velocity for 300ms, and then apply input velocity for 300ms. Visualize the total 2D firing rate $r_{total}(x, y)$ side-by-side at 3 distinct time points (e.g., $t = 300$, $t = 450$, $t = 600$ ms). Describe the integration motion resulting from your velocity input vectors.
Hint: Convolution over the 2D space can be easily implemented using `scipy.ndimage.convolve` and setting the boundary conditions accordingly.

2.4. *(Bonus)* Generate an input pattern with changing velocity. Start with a input velocity pointing in a random direction, with a speed sampled uniformly in $[1, 2]$. Sample a new input velocity every 100 ms, and you could smooth the input velocity when there is a change. Devise a method to compute the velocity of the moving 2D grid pattern. Does the grid velocity track well the input velocity?

Exercise 3. Aperiodic Boundaries

Periodic boundaries (a torus connection topology) are biologically implausible as raw physical tissue does not wrap around itself. The edges of the neural tissue must be considered. We now consider the **aperiodic boundary**: if the source neuron of a connection weight exceeds the boundary, we just omit the connection. Mathematically, we define a strict boundary for integrals in the expression of $I_{rec}(\vec{\mathbf{x}}, t)$:

$$I_{rec}(\vec{\mathbf{x}}, t) = \int_0^M \int_0^M W(\vec{\mathbf{x}} - \vec{\mathbf{x}}')r(\vec{\mathbf{x}}', t)dx'dy' \tag{8}$$

If you are using `scipy.ndimage.convolve`, this could be efficiently implemented by setting `mode='constant'`.

3.1. Repeat the simulation from Ex. 2.2 but with aperiodic boundaries. Describe what you see.

To recover the grid pattern with aperiodic boundaries, there are two proposed methods: (1) attenuate the outgoing weights for neurons near the boundary; (2) attenuate the input for neurons near the boundary. The intuition is that these methods would reduce the influence of artifacts at the boundary. Specifically, we define a radial masking envelope $e(\vec{x})$ that equals 1 in the center and smoothly tapers to 0 near the boundaries:

$$e(\vec{\mathbf{x}}) = \exp(-\gamma^2 \max(0, |\vec{\mathbf{x}}| - R_{fade})^2) \tag{9}$$

where $|\vec{x}|$ is the distance of the neuron at location $\vec{x}$ from the center of the network, $R_{fade} = f_{ratio} \times \frac{M}{2}$ is the radius at which fading begins, and γ configures the exponential fall-off. Use $\gamma = 0.08$ and $f_{ratio} = 0.2$. Because we want to attenuate the effect of boundary, we enlarge the $M \times M$ array of neurons to $M = 50$ in the following simulations.

3.2. Compute and plot the 2D radial masking envelope $e(\vec{\mathbf{x}})$.

3.3. Modify your network such that the outgoing weights of boundary neurons are scaled by $e(\vec{\mathbf{x}})$. (*Hint:* this is equivalent to scale the firing rate of neurons $r_{N,S,E,W}$ by $e(\vec{\mathbf{x}})$ before performing the convolution). Simulate with $\vec{\mathbf{v}} = (0, 0)$ and plot the final network activity $r_{total}(x, y)$. Describe the pattern.

3.4. Instead of attenuating the weight, let's modulate the velocity inputs: multiply B_N, B_S, B_E, B_W by the envelope $e(\vec{\mathbf{x}})$ (making the input position-dependent). Repeat the simulation and plot as in Ex 3.3. Comment on the resulting pattern.

3.5. What is the difference between the two approaches in Ex 3.3 and 3.4? Is it that one works better than the other? Does this have biological consequences/implications?

3.6. *(Bonus)* Let's use the model in Ex 3.4. Generate a input velocity pattern as done in Ex 2.4. Compute the velocity of the moving grid patterns. Does it track well the input velocity? Do you notice the pattern change in any aspect other than just following the velocity input?

Exercise 4. Final Question

Please select **ONLY ONE** of the following three questions and answer it in half a page of free text. Conceptually this is the most challenging part of the miniproject! You need to think deeply and really understand the issues before you answer. You can choose the question that fits best your background and interests. For this final question you are **NOT** allowed to discuss with other teams of students working on the same project.

4.1. [**Theory-oriented**] Assuming that your project works well and has all the qualitative features that you expect it to have. Can you think of a special case of the model for which you can derive mathematically the exact answer, using the mathematical theories covered in class (for example field models)? How could you use this to PRECISELY test your program, to make sure that it is quantitatively correct?
Hint: What type of mismatch would you expect if either the theory calculations or the model simulations are wrong by a factor of 2 at an unknown location? Test your model for quantitative correctness.

4.2. [**Coding-oriented**] This miniproject was inspired by the paper from Burak and Fiete (2009). In the paper, they described their simulation and parameters in the following rephrased paragraph (on the **next page**). What is the EXACT mapping between your code and the description in the paper? If there are differences, what are they?

4.3. [**Biology-oriented**] This miniproject in computational neuroscience should have a link to neurobiology. Be creative and design an experiment that would enable you to test specific aspects of the model predictions. If necessary, think of additional modules or features that you would have to add to your model to make a comparison with experiments possible.

[Coding-oriented] Paper snippet from Burak and Fiete (2009)

The dynamics of rate-based neurons is specified by:

$$\tau \frac{ds_i}{dt} + s_i = f\left(\sum_{j=1}^{N} W_{ij} s_j + B_i\right)$$

The neural transfer function f is a simple rectification nonlinearity: $f(x) = x$ for $x > 0$, and 0 otherwise. The synaptic activation of neuron i is s_i. W_{ij} is the synaptic weight from neuron j to neuron i. The time-constant of neural response is τ= 10 ms. The time-step for numerical integration is $dt = 0.5$ ms. We assume that neurons are arranged in a 2-d sheet. Neuron i is located at x_i. There are $N = 128 \times 128$ neurons in the network. Each neuron i also has a preferred direction (W, N, S, E), denoted as θ_i. Locally, each 2×2 block on the sheet contains one neuron of each preferred direction, tiled uniformly.
The weight W_{ij} from neuron j to neuron i depends on the distance between $\mathbf{x}_i$ and $\mathbf{x}_j$ in the neural sheet:

$$W_{ij} = W_0(\mathbf{x}_i - \mathbf{x}_j - l\hat{\mathbf{e}}_{\theta_j})$$

with

$$W(\mathbf{x}) = ae^{-\gamma|\mathbf{x}|^2} - e^{-\beta|\mathbf{x}|^2}$$

where $W_0(\mathbf{x})$ is a center-surround weight profile, and l is a small shift of the weight profile in the direction $\hat{\mathbf{e}}_{\theta_j}$ (unit vector in the direction of θ_j). In all simulations, we used $a = 1, \gamma = 1.05 \times \beta$ and $\beta = 3/\lambda_{\text{net}}^2$ where $\lambda_{\text{net}} = 13$ is approximately the periodicity of the formed lattice in the neural sheet.
The input B_i to each neuron is given by:

$$B_i = A(x_i)(1 + \alpha \hat{\mathbf{e}}_{\theta_i} \cdot \mathbf{v})$$

where $\mathbf{v}$ is the velocity of the rat. α is a gain factor that determines how the velocity of the rat is translated into the shift of the population pattern in the neural sheet. For the network with periodic boundary conditions, $A(x_i)$ is 1 everywhere. For the aperiodic network:

$$A(\mathbf{x}) = \begin{cases} 1 & |\mathbf{x}| < R - \Delta r \\ \exp\left[-a_0(\frac{|\mathbf{x}|-R+\Delta r}{\Delta r})^2\right] & R - \Delta r < |\mathbf{x}| < R \end{cases}$$

R is the diameter of the network and $a_0 = 4$. In all the aperiodic simulations, we use $\Delta r = R$

Resources

You can find more information about grid cell attractor model and boundary conditions in this paper: Accurate Path Integration in Continuous Attractor Network Models of Grid Cells *by Y. Burak and I. R. Fiete (2009).*
For general biological context regarding the original discovery of grid cells, refer to Microstructure of a spatial map in the entorhinal cortex *by T. Hafting et al. (2005).*